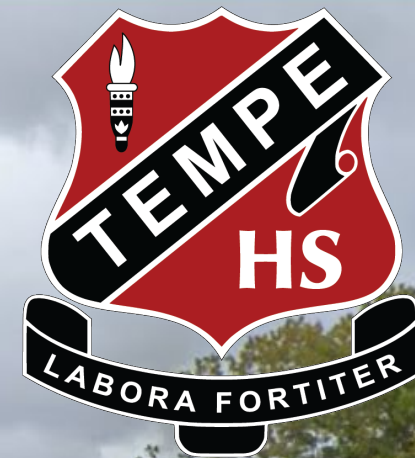




Pi Pico OOP Mini Project

- Ben Jones
- Tempe High School
- Lecture 5



Google

Image capture: Feb 2021



Today

- Stubs & Drivers
- Implement an audio notification Class including a beep
- Write documentation for our Class
- Unit test the audio notification

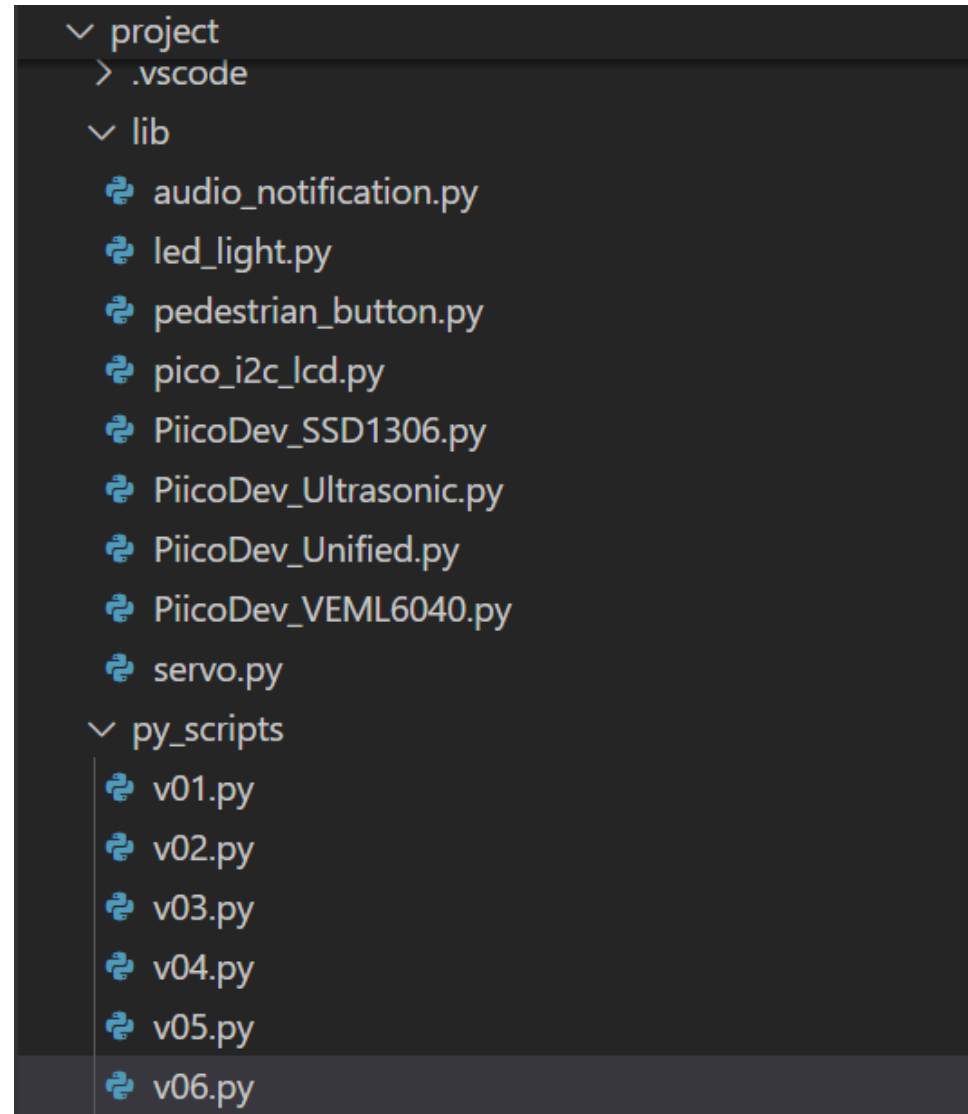




Create new files

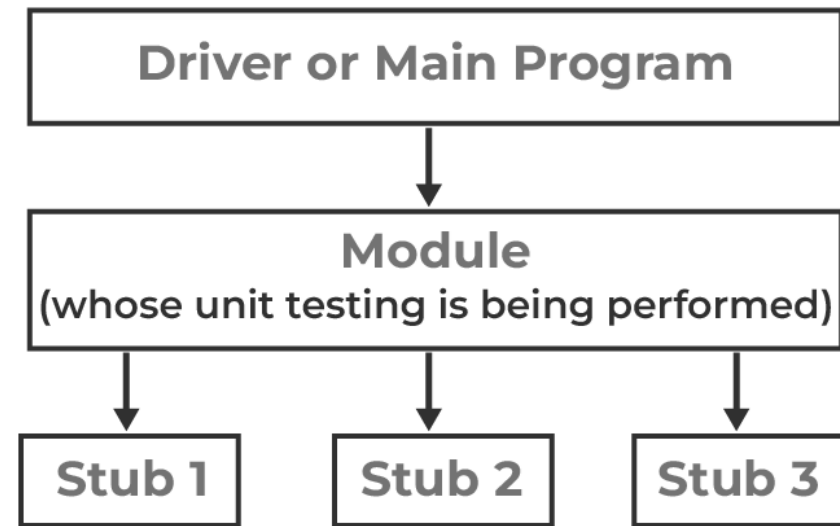
v06.py in project\py_scripts\

audio_notification.py in project\lib\



Stubs & Drivers

Definitions A stub is a simplified version that replaces a real lower component so it doesn't need to be full implemented. A driver is a simple program in a higher system that tests a lower component without fully implementing the higher system.



Implement a stub

```
class Audio_Notification:
    def __init__(self, pin):
        self.__pin = pin

    def warning_on(self):
        # Just pretend to Beep
        print("Beep")
```

Implement a Driver

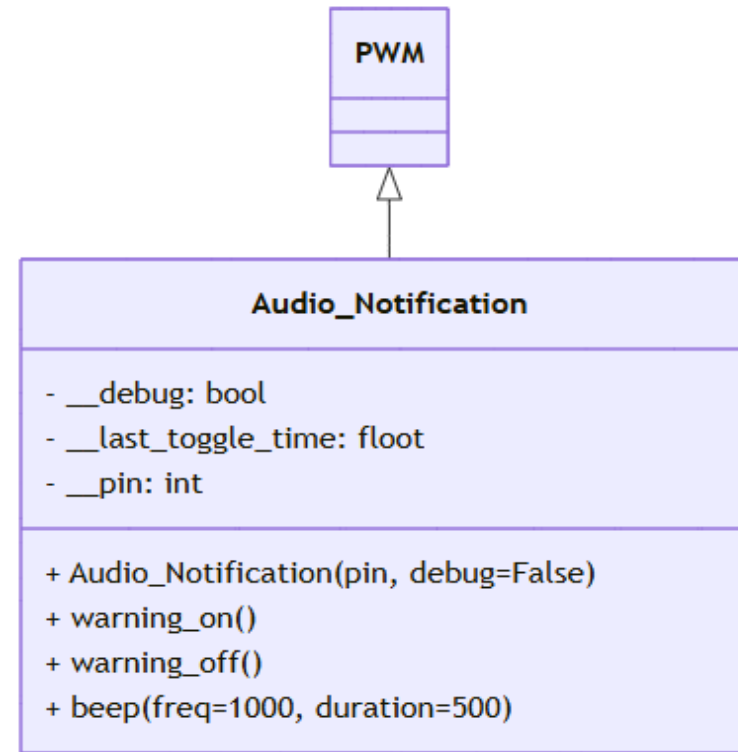
```
from audio_notification import Audio_Notification
from led_light import Led_Light

def subsystem_driver():
    print("Testing Traffic Light...")
    led_pedestrian_green = Led_Light(17, True, True)
    audio_stub = Audio_Notification(27, False)
    # Test walk state
    led_pedestrian_green.on()
    audio_stub.warning_on()
    print("Test complete!")

subsystem_driver()
```



UML





Audio_Notification Imports & Initialiser

```
from machine import Pin, PWM  
from time import sleep, time
```





Audio_Notification Class Initialiser & Attributes

```
class Audio_Notification(PWM):  
def __init__(self, pin, debug=False):  
    super().__init__(Pin(pin))  
    self.__debug = debug  
    self.duty_u16(0) # Start with buzzer off  
    self.__last_toggle_time = time()
```





Audio_Notification .beep() Method

```
def beep(self, freq=1000, duration=500):  
    self.freq(freq)  
    self.duty_u16(32768) # 50% duty cycle  
    sleep(duration / 1000)  
    self.duty_u16(0) # Turn off after beep  
    if self.__debug:  
        print("Beep")
```





Audio_Notification .warning_on() Method

```
def warning_on(self):  
    if self.__debug:  
        print("Warning on")  
    now = time()  
    if now - self.__last_toggle_time >= 0.5:  
        self.beep(freq=500, duration=100)  
        self.__last_toggle_time = now
```





Audio_Notification .warning_off() Method

```
def warning_off(self):  
    if self.__debug:  
        print("Warning off")  
    self.__duty_u16(0) # Turn off sound
```





Add documentation to your Class

Class-level docstring: Briefly describes what the Class represents and lists its attributes and methods.

Constructor docstring: Explains what parameters are needed to create an instance.

Method docstrings: Describe what each method does and what parameters it takes.

```
Class LedPico:
    """
    Simple Class to turn an LED on or off on a Raspberry Pi Pico.

    Args:
        pin (int): GPIO pin number for the LED.

    Example:
        led = LedPico(25)  # Onboard LED
        led.set(True)      # Turn on
        led.set(False)     # Turn off
    """
    def __init__(self, pin):
        from machine import Pin
        self.led = Pin(pin, Pin.OUT)

    def set(self, state: bool):
        """
        Set the LED on or off.

        Args:
            state (bool): True to turn on, False to turn off.
        """
        self.led.value(1 if state else 0)
```





Unit Test the Audio_Notification Class

Your unit test should test all methods, where possible, because it involves an auditory sound you should design a manual test.

Pause the video, design your unit test, then check your unit tests against my suggested batch of unit tests.



```
1 from time import sleep
2 from audio_notification import Audio_Notification
3
4 # Replace 18 with the GPIO pin your buzzer is connected to
5 buzzer = Audio_Notification(18, debug=True)
6
7 print("Testing beep()")
8 buzzer.beep(freq=1000, duration=200)
9 print("Did you hear a beep? (Check your buzzer)")
10
11 print("Testing warning_on() (should beep every ~0.5s for 2 seconds)")
12 start = buzzer._last_toggle_time
13 for _ in range(5):
14     buzzer.warning_on()
15     sleep(0.5)
16
17 print("Testing warning_off() (should silence the buzzer)")
18 buzzer.warning_off()
19 print("Buzzer should now be off.")
20
21 print("Manual test complete.")
```

```
1 def warning_on(self):
2     """
3     Sound a warning beep if 0.5 seconds have elapsed since the last beep.
4
5     This method is designed to be called repeatedly in a main loop to
6     create a periodic warning sound.
7     """
8     if self.__debug:
9         print("Warning on")
10    now = time()
11    if now - self.__last_toggle_time ≥ 0.5:
12        self.beep(freq=500, duration=100)
13        self.__last_toggle_time = now
14
15 def warning_off(self):
16     """
17     Turn off the buzzer sound.
18
19     This stops any active buzzer output by setting the duty cycle to 0.
20     """
21    if self.__debug:
22        print("Warning off")
23    self.duty_u16(0) # Turn off sound
```